

Generative classifiers are just language models: some simple tricks

Jialin Lu 2026-06-08

To sort a sentence into one of three buckets, we now routinely fire up a language model and ask it to write us a paragraph. It is a cannon pointed at a mosquito, and, annoyingly, the cannon is the right tool. This post is about the cannon: why we reach for it, what it is really computing, and three small tricks that stop us from wasting most of its firepower. None of the tricks is new. They are the standard moves for serving any autoregressive language model, and that is the whole point: a generative classifier is one of those, so the tricks drop in for free the moment you stop treating it as a black box. The math is simple and the tricks fall out of it. We measure on CPU, with Qwen3Guard-Gen-0.6B as the running example.

Contents

A cannon for a mosquito	1
What the model is actually computing	2
Where the time goes	3
Three tricks	3
L1: stop decoding	4
L2: aim at the last position only	4
L3: keep the prompt you already paid for	4
What's left	4
So, was the cannon worth it?	5
Appendix: the GPU, and the bigger guns	6
Bibliography	6

A cannon for a mosquito

Here is a slightly absurd thing we do all the time now. To decide whether a piece of text is safe or unsafe, a question with, let's say, three possible answers, we boot up a several-hundred-million-parameter language model and ask it to *write us a sentence*. The model dutifully generates `Safety: Unsafe`, maybe a line of categories after it, and we read the verdict off the front. It works beautifully. It is also a cannon pointed at a mosquito.

The old way to swat the mosquito is a flyswatter: a small classifier. You put a little head on top of an encoder, the encoder turns the input x into a vector $h(x) \in \mathbb{R}^d$, a linear layer of K units turns that into K scores, and a softmax turns the scores into a distribution over your labels,

$$p(y = k \mid x) = \frac{\exp(w_k^\top h(x))}{\sum_{j=1}^K \exp(w_j^\top h(x))}, \quad W \in \mathbb{R}^{K \times d}. \quad (1)$$

Train W and the encoder with cross-entropy on your labelled data and you are done. This is a BERT-style text classifier, it is small, it is fast, and for a great many jobs it is exactly right.¹

So why the cannon? Because lately, especially for content safety, people skip the flyswatter and fine-tune a big pretrained language model to *write the label as text*. Llama Guard², ShieldGemma³, Qwen3Guard⁴, all the same idea. Hand the model a chat prompt and it generates

Safety: Unsafe
Categories: Violent

and you take the word after `Safety:`. Call this a *generative classifier*, as opposed to the discriminative head of Equation 1.

And here is the thing: the cannon really is the right call, for reasons that have nothing to do with speed. A flyswatter trained from scratch knows only the labels you showed it; step outside that distribution and it has nothing to lean on, which is why small fine-tuned classifiers tend to fall apart on inputs that don't look like their training set. The cannon comes pre-loaded. It brings world knowledge, a hundred-odd languages, and a feel for meaning that no 100M-parameter head trained on your labels is going to match. It throws in a bonus, too: the policy lives in the prompt. The list of what counts as unsafe is system-prompt text, not baked into the weights, so changing the policy is an edit, not a retrain.⁵

So we keep the cannon. Fine. What I object to is firing it nine times to kill one mosquito, which, as we'll see, is more or less what the standard recipe does. The rest of this post is about firing it once. First we have to write down, precisely, what the cannon is computing, because once you see it the tricks are obvious.

What the model is actually computing

Let us be exact, because the tricks are just consequences of being exact.

A language model does not own the head from Equation 1. What it owns is a *language-modeling head*: at every position t it produces a distribution over the whole vocabulary $\mathcal{V}$ for the next token,

$$p(x_{t+1} = v \mid x_{\leq t}) = \frac{\exp(E_v^\top h_t)}{\sum_{u \in \mathcal{V}} \exp(E_u^\top h_t)}, \quad E \in \mathbb{R}^{|\mathcal{V}| \times d}. \quad (2)$$

Here h_t is the hidden state at position t and E is the projection from hidden states to vocabulary. The vocabulary is big; for Qwen3 it is about 150,000 tokens. That is the wrong shape for a classifier: a distribution over 150,000 tokens, not over $K = 3$ labels.

The cast is to make the K labels *be* K tokens. `safe`, `unsafe`, `controversial` are each a single token; call their ids i_1, i_2, i_3 . Now set up the prompt so the next token the model wants to write *is* the verdict, read Equation 2 at that one position, and keep only those three logits:

¹ Meta Llama Team. 2024. "Llama-Prompt-Guard-2-86m Model Card."

² Meta Llama Team. 2024. "Llama Guard 3-8b Model Card."

³ Zeng, Wenjun, and others. 2024. "Shield-gemma: Generative AI Content Moderation Based on Gemma."

⁴ Zhao, Haiyang, and others. 2025. "Qwen3guard Technical Report."

Listing 1: The model card's output. Only the word after `Safety:` is the answer; the rest is the model clearing its throat.

⁵ Alibaba ship a "strict" and a "loose" Qwen3Guard with the same weights and different wording. A from-scratch classifier can't do that; changing its label space means training a new one.

$$p(y = k \text{ mid } x) = \frac{\exp(E_{i_k}^\top h_t)}{\sum_{j=1}^K \exp(E_{i_j}^\top h_t)}. \quad (3)$$

In other words: classification is Equation 2, restricted to the label tokens and renormalized. Now put Equation 3 next to Equation 1, and the penny drops. The classification head W we thought we needed is *already in there*, sitting as the three rows $E_{i_1}, E_{i_2}, E_{i_3}$ of the language-modeling matrix. The cannon arrived with a perfectly good classification head bolted on; we just have to read the right three rows, at the right one position.

The chat template and the literal `Safety:` are scaffolding; they push the model into the state where its next token is the verdict. They are not the answer. That's the whole insight, and it is worth boxing, because all three tricks are corollaries of it:

Classification reads the next-token distribution at one position, over K tokens, after a fixed prompt.

Anything the cannon does beyond that one read is, as far as the label is concerned, wasted powder.

Where the time goes

Let's put the waste in milliseconds. The model-card recipe renders the chat prompt, calls `generate()` for a handful of tokens, and parses the text. So one call is a *prefill* pass that encodes the N prompt tokens, then a *decode loop* that spits out the answer one token at a time, and each of those tokens is a full forward pass through the model.

Qwen3Guard writes about nine tokens before the line we care about is finished,⁶ so the decode loop runs roughly nine times. On a small model a forward pass is cheap in arithmetic but not in fixed overhead, and nine of them back to back is most of the wall-clock; on a CPU it is the gap between two seconds and a fifth of one.

Now hold that against the boxed line. We wanted the distribution at *one* position; the decode loop computed nine. We wanted *three* logits; Equation 2 projected onto all 150,000 tokens, at *every* position. We fed a long *fixed* prompt; we re-encoded it from scratch on *every* call. Three kinds of waste. Three tricks, one each.

Three tricks

One disclaimer first, which is really the moral of the whole post: none of what follows is clever. Teacher forcing, reading the logits at the last position, caching

⁶ `Safety:` Unsafe, then the `Categories:` boilerplate it was fine-tuned to always produce.

a fixed prompt's KV; these are the standard, slightly boring optimizations that everyone who serves a language model already uses. The only thing worth saying out loud is that a generative classifier *is* a language model being served, so all three apply to it with no adaptation at all. If your team files the safety model under “classifier” and reaches for it as a black box, the way you would treat an old SVM or a regex, instead of under “autoregressive LM I know how to make fast,” you are leaving a $5 \times$ on the floor for no reason.

With that said: they stack. Each one deletes a piece of generation the classifier doesn't need, and each is just the boxed observation, read back. The savings on our example are in Table 1; we call the rungs L1, L2, L3.

L1: stop decoding

We read the distribution at one position, so generating the rest of the string is pointless.

We already know the model is going to write `Safety:` and then the verdict, so we put the words there ourselves. Append the literal string `Safety:` to the prompt, treat it as input rather than something to be produced, run a *single* forward pass, and read the next-token distribution at the last position. Argmax over the three label logits is the answer. This is teacher forcing; ShieldGemma's card spells out this exact recipe,⁷ and Llama Guard publishes the first-token-logit cousin of it.⁸ There is no decode loop at all: one forward pass instead of ten. This is the big one, because it is the trick that fires the cannon once.

⁷ Zeng, Wenjun, and others. 2024. “Shield-gemma: Generative AI Content Moderation Based on Gemma.”

⁸ Meta Llama Team. 2024. “Llama Guard 3-8b Model Card.”

L2: aim at the last position only

We read *one* position, so projecting every other position onto the vocabulary is wasted aim.

The language-modeling head Equation 2 is a matmul of shape $[N, d] \times [d, |\mathcal{V}|]$: for each of N positions, throw a d -vector at 150,000 logits. But Equation 3 only ever reads the last position. So slice the hidden states from $[N, d]$ down to $[1, d]$ before the projection, and project that one row. The other $N - 1$ projections simply never happen.⁹ This is the same row-reading idea from before, pushed all the way: not just three rows of E , but three rows at one position.¹⁰

⁹ In PyTorch this is `logits_to_keep=1`; ONNX bakes it in as a slice on the graph; llama.cpp and most runtimes already hand back only the last position.

¹⁰ You could also project onto only the three label rows of E instead of all 150,000. On this model it saves under a millisecond, the head being small next to the rest of the pass, so we don't bother and just slice the position.

L3: keep the prompt you already paid for

We classify after a *fixed* prompt, so the work on that prompt is reusable.

The system prompt, the chat template plus the policy text listing the harm categories, is the same on every single call. Inside the model each prompt token produces key and value vectors that later positions attend to; that is the KV cache. Since the prompt prefix never changes, its keys and values never change, so we compute them once and keep them.¹¹ Every call after the first then pays only for the user's content, instead of re-reading the whole policy from the top.

¹¹ Precisely: cache the longest prefix that is identical regardless of user content, and on each call run forward only over the variable suffix.

What's left

The tricks are structural. They change *which* numbers get computed, not how each number is stored, so they carry over to any backend and any precision. We checked on three: PyTorch, ONNX Runtime,¹² and llama.cpp.¹³ The model is Qwen3Guard-Gen-0.6B, batch one, on sixteen cores of a Kunpeng 920¹⁴ aarch64 server CPU. CPU, not GPU, on purpose: for a sub-billion-parameter model at batch one this is the realistic deployment, and it is where each forward pass is expensive enough that the tricks actually buy you something. Each number is the median of 100 timed calls, in milliseconds.

backend	L0 baseline	+L1	+L2	+L3
PyTorch fp32	2148	688	555	408
ONNX fp32	1671	598	485	253
llama.cpp q8_0	643	261	—	129

The shape repeats everywhere: L1 is the cliff, because it kills the decode loop, and L2 and L3 each take another bite. The three tricks on their own walk PyTorch from 2148 down to 408 ms, call it a $5\times$, and the model's output never changes.¹⁵

Two more levers are worth a mention, though neither is one of the three tricks. Quantizing the weights to 8 bits (the llama.cpp q8_0 row) is lossy in principle but almost free in accuracy here, and it drops the best setup to 129 ms. A BLAS kernel tuned to the CPU's microarchitecture trims the fp32 path further. Both stack on top of the tricks rather than standing in for them.

¹² Microsoft. 2018. "ONNX Runtime."

¹³ Gerganov, Georgi, and the llama.cpp contributors. 2023. "Llama.cpp."

¹⁴ Huawei. n.d. "Kunpeng 920 Processor"

Table 1: Per-call latency, p50 ms, walking the trick ladder on three backends. L0 is the model-card `generate()` path. llama.cpp returns only the last position by default, so L2 is already baked into its baseline; that row is also 8-bit quantized, which is why it starts lower.

¹⁵ We gate that claim: on every sample the fast path must return the same verdict as the model-card path, and the fp32 logits must agree to within 10^{-2} . It does.

So, was the cannon worth it?

Yes. You just shouldn't fire it nine times.

A generative classifier is a language model asked to write down a label. Write it out honestly and classification is one thing: the model's next-token distribution, read at one position, over K tokens, after a fixed prompt. That is Equation 3, and its classification head turns out to be three rows of the model's own vocabulary projection. Everything past that read is scaffolding, and the three tricks pull the scaffolding down in order: stop decoding, aim at the last position only, keep the prompt you already paid for. They are precision-neutral, they port across backends, they leave the answer untouched, and together they turn a two-second CPU call into a four-hundred-millisecond one before quantization even enters the picture.

None of the three tricks is an invention. They are exactly what everyone already does to serve a language model fast. The one idea worth carrying away is that a generative classifier is precisely that, a language model being served, so you get all of it for free the moment you stop filing it under "black-box classifier." And the point isn't really about safety classifiers either. Any time you point a generative model at a small, structured answer, most of what it generates is throat-clearing, and throat-clearing is optional.

Appendix: the GPU, and the bigger guns

On a GPU the same forced-prefix trick (L1) is still the one that matters; the decode loop's fixed per-step cost dominates there too. On an RTX 3090, Qwen3Guard-Gen-0.6B with L1 runs about 29 ms p50 against 237 ms for the model-card path, at a representative input length. L2 and L3 barely register on the GPU, since its vocabulary projection and prompt re-encoding are cheap next to launch overhead. That is the mirror image of the CPU story, and the reason the CPU is the more interesting place to run the full ladder.

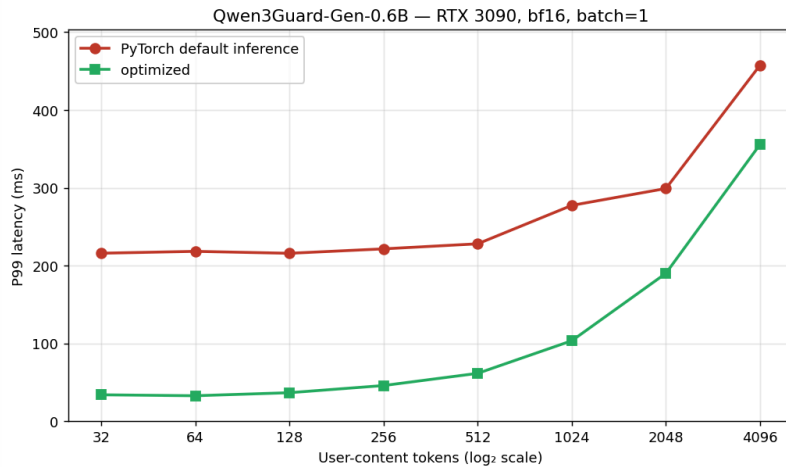


Figure 1: Qwen3Guard-Gen-0.6B on an RTX 3090: the model-card path versus the forced-prefix path (L1), p99 latency across input lengths. The forced-prefix path stays under a 200 ms budget out to about 2048 input tokens.

The same cast works on the bigger members of the family.

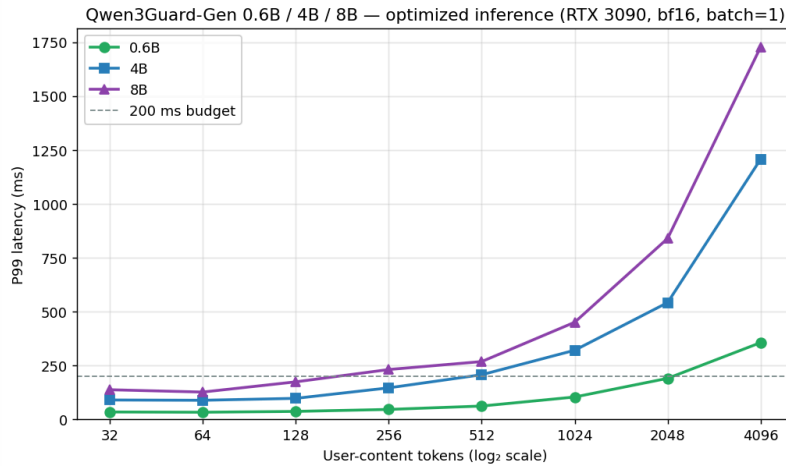


Figure 2: The forced-prefix path across the three Qwen3Guard-Gen sizes (0.6B / 4B / 8B) on an RTX 3090. All three get the same trick; the 0.6B clears a 200 ms budget at the longest inputs, the bigger two bind earlier.

Bibliography

Gerganov, Georgi, and the llama.cpp contributors. 2023. "Llama.cpp."

Huawei. n.d. "Kunpeng 920 Processor."

Meta Llama Team. 2024. "Llama Guard 3-8b Model Card."

Meta Llama Team. 2024. "Llama-Prompt-Guard-2-86m Model Card."

Microsoft. 2018. "ONNX Runtime."

Zeng, Wenjun, and others. 2024. "Shieldgemma: Generative AI Content Moderation Based on Gemma."

Zhao, Haiyang, and others. 2025. "Qwen3guard Technical Report."